

AG35-CET&AG35-EUT

QuecOpen(SDK)语音服务开发指导

LTE Standard 模块系列

版本：1.0.1

日期：2025-03-24

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2025.

文档历史

修订记录

版本	日期	作者	变更表述
-	2023-10-24	Sunshine HUANG	文档创建
1.0.0	2023-10-24	Sunshine HUANG	临时版本
1.0.1	2025-03-24	Joetar CHEN	临时版本 1. 新增使用模块 AG35-EUT。 2. 新增 API <code>ql_voice_set_service_error_cb()</code> （第 2 章）。

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 语音服务 API.....	8
2.1. 头文件.....	8
2.2. 函数概览.....	8
2.3. 函数详解.....	9
2.3.1. ql_voice_init.....	9
2.3.2. ql_voice_deinit	10
2.3.3. ql_voice_dial.....	10
2.3.4. ql_voice_hangup_all	11
2.3.5. ql_voice_answer	11
2.3.6. ql_voice_hangup	11
2.3.7. ql_voice_hold	12
2.3.8. ql_voice_unhold	12
2.3.9. ql_voice_get_records	13
2.3.9.1. ql_voice_record_array_t.....	13
2.3.9.2. ql_voice_record_t.....	14
2.3.9.3. QL_VOICE_STATE_E.....	14
2.3.10. ql_voice_forwarding	15
2.3.10.1. QL_VOICE_FW_COND_E.....	16
2.3.11. ql_voice_get_forwarding_status	17
2.3.11.1. ql_voice_fw_status_t.....	17
2.3.12. ql_voice_waiting.....	18
2.3.13. ql_voice_get_waiting_status	18
2.3.14. ql_voice_autoanswer	19
2.3.15. ql_voice_send_dtmf_char	19
2.3.16. ql_voice_set_call_cb.....	20
2.3.16.1. ql_voice_call_cb_f.....	20
2.3.17. ql_voice_set_service_error_cb	21
2.3.17.1. ql_voice_service_error_cb_f.....	21
3 示例.....	22
3.1. 拨打电话	22
3.2. 挂断电话	23
3.3. 接听电话	23
3.4. 挂断所有拨打的电话.....	24
3.5. 呼叫保持	24
3.6. 获取通话记录.....	25
3.7. 呼叫转移	26

3.8.	呼叫等待	27
3.9.	自动接听电话.....	28
3.10.	发送 DTMF 字符	29
4	附录 参考文档及术语缩写	30

Preliminary
Quectel Confidential

表格索引

表 1: 函数概览	8
表 2: 参考文档	30
表 3: 术语缩写	30

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍在 QuecOpen®方案下，AG35-CET 和 AG35-EUT 模块的语音服务 API 及相关示例。

Preliminary
Quectel Confidential

2 语音服务 API

2.1. 头文件

语音服务 API 头文件为 `ql_voice.h`，位于 SDK 的 `ql-sysroots/usr/include/ql-sdk/`目录下。若无特别说明，本文档所涉及头文件均在该目录下。

2.2. 函数概览

表 1：函数概览

函数	说明
<code>ql_voice_init()</code>	初始化语音服务
<code>ql_voice_deinit()</code>	注销语音服务
<code>ql_voice_dial()</code>	拨打电话
<code>ql_voice_hangup_all()</code>	挂断所有拨打的电话
<code>ql_voice_answer()</code>	接听电话
<code>ql_voice_hangup()</code>	挂断电话
<code>ql_voice_hold()</code>	开启呼叫保持
<code>ql_voice_unhold()</code>	关闭呼叫保持
<code>ql_voice_get_records()</code>	获取通话记录
<code>ql_voice_forwarding()</code>	设置呼叫转移
<code>ql_voice_get_forwarding_status()</code>	查询呼叫转移状态
<code>ql_voice_waiting()</code>	设置呼叫等待

<code>ql_voice_get_waiting_status()</code>	查询呼叫等待状态
<code>ql_voice_autoanswer()</code>	设置自动接听
<code>ql_voice_send_dtmf_char()</code>	发送 DTMF 字符
<code>ql_voice_set_call_cb()</code>	注册电话信息接收回调函数
<code>ql_voice_set_service_error_cb()</code>	设置语音服务错误事件接收回调函数

备注

若无特别说明，本文档所述函数均不支持并发调用，并且不能在相关回调函数中调用以上函数。

2.3. 函数详解

2.3.1. ql_voice_init

该函数用于初始化语音服务。

- 函数原型

```
int ql_voice_init(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

备注

使用其他语音服务 API 前，必须调用本函数初始化语音服务。

2.3.2. ql_voice_deinit

该函数用于注销语音服务。

- 函数原型

```
int ql_voice_deinit(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

若不再使用语音服务，必须调用本函数注销语音服务、释放资源。

2.3.3. ql_voice_dial

该函数用于拨打电话。

- 函数原型

```
int ql_voice_dial(char *num, int len, uint32_t *id)
```

- 参数

num:
[In] 对方电话号码。

len:
[In] 电话号码长度。

id:
[Out] 本次通话 ID。用于后续通话操作。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.4. ql_voice_hangup_all

该函数用于挂断所有拨打的电话。

- 函数原型

```
int ql_voice_hangup_all(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.5. ql_voice_answer

该函数用于接听电话。

- 函数原型

```
int ql_voice_answer(uint32_t id)
```

- 参数

id:
[In] 通话 ID。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.6. ql_voice_hangup

该函数用于挂断电话。

- 函数原型

```
int ql_voice_hangup(uint32_t id)
```

- 参数

id:

[In] 通话 ID。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.7. ql_voice_hold

该函数用于开启呼叫保持。

- 函数原型

```
int ql_voice_hold(uint32_t id)
```

- 参数

id:

[In] 通话 ID。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*

备注

须确保已向运营商订购了呼叫保持业务，方可开启呼叫保持功能。

2.3.8. ql_voice_unhold

该函数用于关闭呼叫保持。

- 函数原型

```
int ql_voice_unhold(uint32_t id)
```

- 参数

id:

[In] 通话 ID。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

备注

确保已向运营商订购了呼叫保持业务后，才能关闭呼叫保持功能。

2.3.9. ql_voice_get_records

该函数用于获取通话记录。

- 函数原型

```
int ql_voice_get_records(ql_voice_record_array_t *p_arr)
```

- 参数

`p_arr`:
[Out] 通话记录表；详见第2.3.9.1章。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.9.1. ql_voice_record_array_t

通话记录表结构体定义如下：

```
typedef struct
{
    int len;
    ql_voice_record_t records[QL_VOICE_MAX_RECORDS];
} ql_voice_record_array_t;
```

- 参数

类型	参数	描述
int	<i>len</i>	通话记录个数

<code>ql_voice_record_t</code>	<code>records</code>	通话记录详情。由参数 <code>len</code> 指定个数， <code>QL_VOICE_MAX_RECORDS</code> 表示可保存的最大记录，最大值为 8；详见第 2.3.9.2 章。
--------------------------------	----------------------	--

2.3.9.2. ql_voice_record_t

通话记录结构体定义如下：

```
typedef struct
{
    uint32_t          id;
    char              number[QL_VOICE_MAX_PHONE_NUMBER + 1];
    QL_VOICE_STATE_E  state;
    QL_VOICE_TECH_E   tech;
    QL_VOICE_DIR_E    dir;
    QL_VOICE_END_REASON_E  end_reason;
} ql_voice_record_t;
```

● 参数

类型	参数	描述
<code>uint32_t</code>	<code>id</code>	通话 ID
<code>char</code>	<code>number</code>	电话号码
<code>QL_VOICE_STATE_E</code>	<code>state</code>	当前通话状态；详见第 2.3.9.3 章。
<code>QL_VOICE_TECH_E</code>	<code>tech</code>	语音服务遵循的技术规范。 <code>QL_VOICE_TECH_3GPP</code> 遵循 3GPP 规范 <code>QL_VOICE_TECH_3GPP2</code> 遵循 3GPP2 规范
<code>QL_VOICE_DIR_E</code>	<code>dir</code>	通话方向。 <code>QL_VOICE_DIR_MO</code> 主叫 <code>QL_VOICE_DIR_MT</code> 被叫
<code>QL_VOICE_END_REASON_E</code>	<code>end_reason</code>	电话挂断原因。 挂断原因比较多，详见 <code>ql_voice.h</code> 头文件。

2.3.9.3. QL_VOICE_STATE_E

当前通话状态枚举定义如下：

```
typedef enum
{
```

```
QL_VOICE_STATE_INCOMING = 0x0000,
QL_VOICE_STATE_DIALING  = 0x0001,
QL_VOICE_STATE_ALERTING = 0x0002,
QL_VOICE_STATE_ACTIVE   = 0x0003,
QL_VOICE_STATE_HOLDING  = 0x0004,
QL_VOICE_STATE_END      = 0x0005,
QL_VOICE_STATE_WAITING  = 0x0006,
} QL_VOICE_STATE_E;
```

- 成员

成员	描述
QL_VOICE_STATE_INCOMING	有电话等待接入
QL_VOICE_STATE_DIALING	拨号中
QL_VOICE_STATE_ALERTING	等待被叫接听
QL_VOICE_STATE_ACTIVE	电话已接听，通话建立。
QL_VOICE_STATE_HOLDING	呼叫保持中
QL_VOICE_STATE_END	通话结束
QL_VOICE_STATE_WAITING	呼叫等待中

2.3.10. ql_voice_forwarding

该函数用于设置呼叫转移。

- 函数原型

```
int ql_voice_forwarding(int reg, QL_VOICE_FW_COND_E cond, char *num, int len)
```

- 参数

reg:

[In] 呼叫转移开启状态。

0 关闭

其他值 开启

cond:

[In] 呼叫转移的条件；详见第2.3.10.1章。

num:

[In] 呼叫转移的电话号码。

len:

[In] 呼叫转移的电话号码长度。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败；错误码详见 *ql_type.h*

备注

须确保已向运营商订购了呼叫转移业务，方可开启或关闭呼叫转移功能。

2.3.10.1. QL_VOICE_FW_COND_E

呼叫转移条件枚举定义如下：

```
typedef enum
{
    QL_VOICE__FW_COND_UNCONDITIONALLY = 0,
    QL_VOICE__FW_COND_MOBILEBUSY      = 1,
    QL_VOICE__FW_COND_NOREPLY          = 2,
    QL_VOICE__FW_COND_UNREACHABLE      = 3,
    QL_VOICE__FW_COND_ALLFORWARDING    = 4,
    QL_VOICE__FW_COND_ALLCONDITIONAL   = 5,
} QL_VOICE_FW_COND_E;
```

- 成员

成员	描述
QL_VOICE__FW_COND_UNCONDITIONALLY	无条件转移
QL_VOICE__FW_COND_MOBILEBUSY	条件转移：被叫忙时转移。
QL_VOICE__FW_COND_NOREPLY	条件转移：被叫无回应时转移。
QL_VOICE__FW_COND_UNREACHABLE	条件转移：被叫不可达时转移（例如被叫关机）。
QL_VOICE__FW_COND_ALLFORWARDING	全部转移。即无条件转移和条件转移。
QL_VOICE__FW_COND_ALLCONDITIONAL	所有的条件转移

2.3.11. ql_voice_get_forwarding_status

该函数用于查询呼叫转移状态。

● 函数原型

```
int ql_voice_get_forwarding_status(QL_VOICE_FW_COND_E cond, ql_voice_fw_status_t *p_status)
```

● 参数

cond:

[In] 呼叫转移的条件；详见第2.3.10.1章。不可将 `QL_VOICE__FW_COND_ALLFORWARDING` 和 `QL_VOICE__FW_COND_ALLCONDITIONAL` 作为查询条件。

p_status:

[Out] 呼叫转移状态信息；详见第2.3.11.1章。

● 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.11.1. ql_voice_fw_status_t

呼叫转移状态信息结构体定义如下：

```
typedef struct
{
    int enabled;
    int len;
    struct {
        QL_VOICE_FW_TYPE_E type;
        char number[QL_VOICE_MAX_PHONE_NUMBER + 1];
    } details[QL_VOICE_MAX_FW_DETAIL_LENGTH];
} ql_voice_fw_status_t;
```

● 参数

类型	参数	描述
int	<i>enabled</i>	呼叫转移开启状态。
		0 关闭
		1 开启

int	len	呼叫转移个数	
QL_VOICE_FW_TYPE_E	type	呼叫转移类型。	
		QL_VOICE_FW_TYPE_VOICE	语音转移
		QL_VOICE_FW_TYPE_DATA	数据转移
		QL_VOICE_FW_TYPE_VOICE_DATA	语音数据转移
char	number	呼叫转移号码	
anonymous struct	details	呼叫转移详细信息。详见 <i>type</i> 和 <i>number</i> 参数定义。	

2.3.12. ql_voice_waiting

该函数用于设置呼叫等待。

- 函数原型

```
int ql_voice_waiting(int enable)
```

- 参数

enable:

[In] 呼叫等待开启状态。

- 1 开启
- 0 关闭

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

须确保已向运营商订购了呼叫等待业务，方可开启或关闭呼叫等待功能。

2.3.13. ql_voice_get_waiting_status

该函数用于查询是否已开启呼叫等待功能。

- 函数原型

```
int ql_voice_get_waiting_status(int *enabled)
```

- 参数

enabled:

[Out] 呼叫等待开启状态。

- 1 开启
- 0 关闭

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.14. ql_voice_autoanswer

该函数用于设置自动接听。

- 函数原型

```
int ql_voice_autoanswer(int enable, uint32_t sec)
```

- 参数

enable:

[In] 自动接听开启状态。

- 1 开启
- 0 关闭

sec:

[In] 等待时间。单位：秒。等待设置的时间值后，自动接听电话。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.15. ql_voice_send_dtmf_char

该函数用于发送 DTMF 字符。

- 函数原型

```
int ql_voice_send_dtmf_char(uint32_t id, char c)
```

- 参数

id:

[In] 通话 ID。

c:

[In] DTMF 字符。合法字符：0~9、A~D、* 和 #。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.16. ql_voice_set_call_cb

该函数用于注册通话信息接收回调函数。

- 函数原型

```
int ql_voice_set_call_cb(ql_voice_call_cb_f cb)
```

- 参数

cb:

[In] 通话信息接收回调函数；详见第 2.3.16.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.16.1. ql_voice_call_cb_f

该回调函数接收通话信息。

- 函数原型

```
typedef void (*ql_voice_call_cb_f)(ql_voice_record_array_t *p_arr)
```

- 参数

p_arr:

[Out] 通话记录表；详见第 2.3.9.1 章。

2.3.17. ql_voice_set_service_error_cb

该函数用于设置语音服务错误事件接收回调函数。仅当语音服务异常时，该回调函数才会被执行。

- 函数原型

```
int ql_voice_set_service_error_cb(ql_voice_service_error_cb_f cb)
```

- 参数

cb:

[In] 语音服务错误事件接收回调函数；详见第2.3.17.1章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.17.1. ql_voice_service_error_cb_f

该回调函数接收语音服务错误事件。

- 函数原型

```
typedef void (*ql_voice_service_error_cb_f)(int error);
```

- 参数

error:

[Out] 语音服务错误码；详见 *ql_type.h*。

- 返回值

无

3 示例

本章所述代码示例均摘自 `sample/voice/main.c`，参考示例时请注意以下几点：

1. 程序启动后，必须调用 `ql_voice_init()` 初始化语音服务；
2. 程序退出前或不再使用语音服务时，必须调用 `ql_voice_deinit()` 释放资源；
3. 使用呼叫保持、呼叫等待、呼叫转移功能时，须确保已向运营商订购了相应业务。

3.1. 拨打电话

```
static void show_ql_voice_dial(void)
{
    int ret = 0;
    uint32_t id;
    char number[QL_VOICE_MAX_PHONE_NUMBER] = {0};

    printf("show ql_voice_dial: \n");
    printf("please enter the phone number: ");
    scanf("%s", number);
    getchar();
    ret = ql_voice_dial(number, strlen(number), &id);
    if (ret == QL_ERR_OK)
    {
        printf("ok, phone number is %s, call id is %u\n", number, id);
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.2. 挂断电话

```
static void show_ql_voice_hangup(void)
{
    int ret = 0;
    uint32_t id;

    printf("show ql_voice_hangup: \n");
    printf("please enter call id: ");
    scanf("%u", &id);
    getchar();
    printf("call is %u\n", id);
    ret = ql_voice_hangup(id);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.3. 接听电话

```
static void show_ql_voice_answer(void)
{
    int ret = 0;
    uint32_t id;

    printf("show ql_voice_answer: \n");
    printf("please enter call id: ");
    scanf("%u", &id);
    getchar();
    printf("call is %u\n", id);
    ret = ql_voice_answer(id);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
}
```

```

else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.4. 挂断所有拨打的电话

```

static void show_ql_voice_hangup_all(void)
{
    int ret = 0;

    printf("show ql_voice_hangup_all: ");

    ret = ql_voice_hangup_all();
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}

```

3.5. 呼叫保持

```

static void show_ql_voice_hold(void)
{
    int ret = 0;
    uint32_t id;

    printf("show ql_voice_hold: \n");
    printf("please enter call id: ");
    scanf("%u", &id);
    getchar();
    printf("call is %u\n", id);

    ret = ql_voice_hold(id);
}

```

```

if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.6. 获取通话记录

```

static void show_ql_voice_get_records(void)
{
    int i;
    int ret;
    ql_voice_record_array_t arr = {0};

    printf("show ql_voice_get_records: ");
    ret = ql_voice_get_records(&arr);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
        return;
    }

    if (0 == arr.len)
    {
        printf("No records\n");
        return;
    }

    for (i = 0; i < arr.len; i++)
    {
        printf("  call id: %u\n", arr.records[i].id);
        printf("  number: %s\n", arr.records[i].number);
        if (arr.records[i].state <= QL_VOICE_STATE_WAITING

```

```

        && arr.records[i].state >= QL_VOICE_STATE_INCOMING)
    {
        printf("  state: %s\n", voice_state_str[arr.records[i].state]);
    }
    else
    {
        printf("  state: unknown %d\n", arr.records[i].state);
    }
    if (arr.records[i].tech <= QL_VOICE_TECH_3GPP2
        && arr.records[i].tech >= QL_VOICE_TECH_NONE)
    {
        printf("  tech: %s\n", voice_tech_str[arr.records[i].tech]);
    }
    else
    {
        printf("  tech: unknown %d\n", arr.records[i].tech);
    }

    printf("  call direction: %d\n", arr.records[i].dir);
    printf("  end reason: %d\n\n", arr.records[i].end_reason);
}
}

```

3.7. 呼叫转移

```

static void show_ql_voice_forwarding(void)
{
    int ret;
    int reg = 0;
    QL_VOICE_FW_COND_E cond;
    char num[QL_VOICE_MAX_PHONE_NUMBER] = {0};

    printf("show ql_voice_forwarding: \n");
    printf("register forwarding? (0-no, 1-yes): ");
    scanf("%d", &reg);
    getchar();
    printf("under what condition?\n");
    printf(" 0: Unconditional\n");
    printf(" 1: Forward when the mobile device is busy\n");
    printf(" 2: Forward when there is no reply\n");
    printf(" 3: Forward when the call is unreachable\n");
}

```

```

printf(" 4: All forwarding(0-3)\n");
printf(" 5: All conditional forwarding(1-3)\n");
scanf("%d", (int *)&cond);
if (cond < 0 || cond > 5)
{
    printf("bad choice\n");
    return;
}
getchar();
if (0 == reg)
{
    ret = ql_voice_forwarding(reg, cond, NULL, 0);
}
else
{
    printf("enter the phone number: ");
    scanf("%s", num);
    getchar();

    ret = ql_voice_forwarding(reg, cond, num, strlen(num));
}

if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.8. 呼叫等待

```

static void show_ql_voice_waiting(void)
{
    int enable;
    int ret = 0;

    printf("show ql_voice_waiting: \n");
}

```

```

printf("enable(0-no, 1-yes): ");
scanf("%d", &enable);
getchar();

ret = ql_voice_waiting(enable);
if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.9. 自动接听电话

```

static void show_ql_voice_autoanswer(void)
{
    int enable;
    uint32_t sec;
    int ret = 0;

    printf("show ql_voice_autoanswer: \n");
    printf("enable(0-no, 1-yes): ");
    scanf("%d", &enable);
    getchar();
    printf("seconds before auto answer: ");
    scanf("%u", &sec);
    getchar();

    ret = ql_voice_autoanswer(enable, sec);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}

```

3.10. 发送 DTMF 字符

```
static void show_ql_voice_send_dtmf_char(void)
{
    int ret = 0;
    uint32_t id;
    char c;

    printf("show ql_voice_send_dtmf_char: \n");
    printf("please enter call id: ");
    scanf("%u", &id);
    getchar();
    printf("please enter dtmf char('0~9', 'A~d', '*', '#'): ");
    scanf("%c", &c);
    getchar();

    ret = ql_voice_send_dtmf_char(id, c);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

4 附录 参考文档及术语缩写

表 2: 参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导

表 3: 术语缩写

缩写	英文描述	中文描述
ACK	Acknowledgment	确认消息
API	Application Programming Interface	应用程序编程接口
DTMF	Dual-tone Multifrequency	双音多频
ID	Identity	身份标识号
IoV	Internet of Vehicles	车联网
MSD	Minimum Set of Data	最小数据集
SDK	Software Development Kit	软件开发工具包
3GPP	3rd Generation Partnership Project	第三代合作伙伴计划